Neural Polyhedral Cost Modeling & Zero-Latency In-Tree LLVM Compiler Optimizer

Lakshya Kohli
Indian Institute of Technology, Varanasi
Varanasi, India
lakshyakohli23@gmail.com

Pranshu Goyal
Indian Institute of Technology, Varanasi
Varanasi, India
pranshu.goyal71@gmail.com

***Abstract*—Cost models in production optimizing compilers face a fundamental trade-off: classical polyhedral heuristics rely on rigid analytical formulations that fail to capture modern microarchitectural non-linearities, whereas deep neural surrogates introduce unacceptable compilation latency overheads and out-of-distribution performance regressions. We present pRNG, an in-tree polyhedral cost modeling framework directly integrated into the LLVM middle-end optimization pipeline. pRNG introduces a hybrid predictive engine combining recursive Tree-LSTM structural abstract syntax tree (AST) embeddings, 35-dimensional ScalarEvolution microarchitectural feature extraction, and pass-specialized Gradient Boosted Decision Tree (GBDT) ensembles. By transpiling trained models into native C/C++ inline branch tables and enforcing epistemic confidence safety gating (τ), pRNG eliminates external runtime dependencies and suppresses offline holdout slowdown risks from 46.49% to 2.23% across 60,220 schedules. Evaluated on physical host execution against upstream Stock Clang `-O3` and GNU GCC `-O3`, pRNG achieves a +13.3% geometric mean speedup (+37.3% on 5-D tensor contractions, +35.5% on polyhedral loop nests) over Stock Clang, with hardware performance counter profiling confirming a 75.0% reduction in L1-D cache misses and 100% vector register spill freedom.**

***Index Terms*—LLVM Compiler Infrastructure, Polyhedral Loop Optimization, Machine Learning Cost Model, Tree-LSTM, Epistemic Uncertainty, Microarchitectural Profiling.**

I. Introduction

The execution efficiency of loop-intensive compute kernels across linear algebra, image processing, and tensor contractions depends decisively on loop transformations such as multi-level tiling, loop interchange, unroll-and-jam, and loop fusion. Modern production compilers, including LLVM's Polly infrastructure [1], [2] and GCC's Graphite engine, rely on classical polyhedral cost models. While geometrically sound for legality analysis, analytical heuristics depend on idealized hardware assumptions (e.g., linear cache access costs, infinite register files, and uniform stride penalties). On modern microarchitectures characterized by non-linear cache eviction policies, aggressive prefetchers, and out-of-order execution pipelines, static heuristics frequently commit unprofitable transformation schedules, causing cache thrashing and execution stalls.

Conversely, deep neural cost models (such as Graph Neural Networks and Transformers) achieve high offline ranking correlation but remain impractical for production compilation. They introduce millisecond-scale inference overheads, require heavy external runtimes (such as PyTorch or ONNX), and suffer from catastrophic out-of-distribution performance regressions on unseen loop topologies.

To address this challenge, we present **pRNG**, an in-tree polyhedral cost model and optimization framework integrated into LLVM 18/20. pRNG operates through a carefully calibrated, production-oriented design:

1) **Structural AST Modeling via Tree-LSTM:** A recursive Tree-LSTM network captures nested loop dependencies, schedule depth, and schedule topologies from Tiramisu polyhedral intermediate representations [3].
2) **Microarchitectural Physical Feature GBDT Experts:** A 35-dimensional physical feature extractor querying LLVM ScalarEvolution (`SCEV`) and Alias Analysis, coupled with pass-specialized Gradient Boosted Decision Tree (GBDT) ensembles.
3) **Native In-Tree C/C++ Transpilation:** Trained GBDT models are transpiled directly into flat, static C/C++ nested `constexpr` branch tables, requiring zero dynamic memory allocation and minimizing compilation overhead ($< 15\,\mu$s).
4) **Epistemic Uncertainty & Safety Gating:** A dynamic variance gating mechanism (τ) that detects uncertain predictions and automatically falls back to stock `-O3` compiler heuristics, suppressing regression risk.

Table I summarizes how pRNG compares against existing compiler cost modeling paradigms.

II. Background & Related Work

A. Polyhedral Framework and Schedule Spaces

In the polyhedral model, loop iterations are represented as integer points inside a parametric polyhedron $\mathcal{D} = \{\vec{i} \in \mathbb{Z}^n \mid A\vec{i} + \vec{b} \geq \vec{0}\}$. Loop transformations (tiling, unrolling, interchange, skewing, fusion) are represented as affine schedule relations $\theta : \mathcal{D} \to \mathcal{T}$. While polyhedral representations ensure transformation legality, finding the optimal schedule within vast combinatorial schedule spaces ($\sim 10^6$ variants per loop nest) requires an accurate profitability cost function.

B. Machine Learning for Compiler Optimization

Early machine learning compilers applied linear regression or flat random forests on hand-crafted loop count features. Recent approaches explore Graph Neural Networks (GNNs)

TABLE I
COMPARISON OF LOOP COST MODELING PARADIGMS IN COMPILERS

Cost Model Paradigm	Microarch. Awareness	Compiler Integration	Compilation Overhead	Out-of-Distribution Safety	Polyhedral Support
LLVM Polly (ISL Heuristic) [2]	Low (Linear cache assumption)	Native LLVM Pass	Negligible ($< 1\,\mu$s)	High (Conservative)	Full Polyhedral
GCC Graphite (ISL Analytical)	Low (Analytical stride count)	Native GCC Pass	Negligible ($< 1\,\mu$s)	High (Conservative)	Polyhedral SCoP
Deep Neural Networks / GNNs	High (Implicit representation)	Out-of-Process Python/IPC	High (50 ms–2 s)	Low (Unbounded regressions)	Limited AST / IR
pRNG (This Work)	**High (35-D SCEV + AST)**	**Native In-Tree C++**	**Minimal ($< 15\,\mu$s)**	**High (Epistemic Gate τ)**	**Full Tiramisu/Polly**

or Transformer models over compiler IR. However, neural compilers face two major barriers: (1) heavy inference overheads that render production builds unacceptably slow, and (2) high sensitivity to out-of-distribution loops, causing severe performance regressions on production workloads.

III. THE FOUR EVOLUTIONARY STAGES OF PRNG

The design of pRNG underwent four rigorous iterations to resolve accuracy, stability, and compilation latency constraints, summarized in Table II.

1) **Stage 1: Tabular Baselines:** Initially, we evaluated standard tabular models (Random Forests, LightGBM) trained on basic loop statistics (trip counts, nest depths). These models achieved an inadequate Spearman rank correlation ($\rho = 0.3140$) and triggered a 32.1% slowdown rate, demonstrating that flat feature vectors cannot model deep loop hierarchies.
2) **Stage 2: Sequential Deep Learning:** We replaced tabular models with 1D-LSTM recurrent networks over serialized AST token streams. While capturing sequential dependencies, 1D-LSTMs failed to represent multi-dimensional loop nests ($\rho = 0.3890$, 29.5% slowdowns).
3) **Stage 3: Hybrid Tree-LSTM & GBDT:** We introduced a dual-stream architecture pairing a recursive Tree-LSTM (capturing AST structural topology) with pass-specialized GBDT experts (capturing exact microarchitectural features), boosting ranking correlation to $\rho = 0.6240$.
4) **Stage 4: Production In-Tree Integration with Safety Gating:** To deploy pRNG into LLVM, we replaced heavy Python/PyTorch inference with native C/C++ transpiled branch tables and introduced epistemic confidence safety gating (τ). This eliminated external runtime dependencies, achieved $\rho = 0.7013$ on 60,220 holdout schedules, and suppressed holdout slowdown risk to 2.23%.

IV. HYBRID PREDICTIVE ENGINE DESIGN

The pRNG architecture unifies three core components:

1) **Recursive Tree-LSTM Structural Representation:** For an AST node j with child set $C(j)$, the hidden state $\mathbf{h}_j$ and memory cell $\mathbf{c}_j$ are computed recursively:

$$\tilde{\mathbf{h}}_j = \sum_{k \in C(j)} \mathbf{h}_k, \quad \mathbf{i}_j = \sigma\left(W^{(i)}\mathbf{x}_j + U^{(i)}\tilde{\mathbf{h}}_j + \mathbf{b}^{(i)}\right) \tag{1}$$

$$\mathbf{f}_{jk} = \sigma\left(W^{(f)}\mathbf{x}_j + U^{(f)}\mathbf{h}_k + \mathbf{b}^{(f)}\right) \tag{2}$$

$$\mathbf{u}_j = \tanh\left(W^{(c)}\mathbf{x}_j + U^{(c)}\tilde{\mathbf{h}}_j + \mathbf{b}^{(c)}\right) \tag{3}$$

$$\mathbf{c}_j = \mathbf{i}_j \odot \mathbf{u}_j + \sum_{k \in C(j)} \mathbf{f}_{jk} \odot \mathbf{c}_k \tag{4}$$

$$\mathbf{h}_j = \sigma\left(W^{(o)}\mathbf{x}_j + U^{(o)}\tilde{\mathbf{h}}_j + \mathbf{b}^{(o)}\right) \odot \tanh(\mathbf{c}_j) \tag{5}$$

2) **Pass-Specialized GBDT Mixture-of-Experts:** Rather than employing a single monolithic model, pRNG trains distinct GBDT models specialized for specific loop transformations: (1) Loop Tiling Expert, (2) Loop Unrolling & Jamming Expert, and (3) Loop Fusion & Skewing Expert. Predictions are blended with the Tree-LSTM structural embedding:

$$\hat{y}_{\text{blend}}(\mathbf{x}) = \alpha \cdot \hat{y}_{\text{spec}}(\mathbf{x}) + (1 - \alpha) \cdot \hat{y}_{\text{LSTM}}(\mathbf{x}) \tag{6}$$

where $\alpha = 0.80$ leverages physical microarchitectural features while retaining structural regularization.
3) **Epistemic Uncertainty Estimation & Safety Gating (Strategy 3):** We estimate prediction uncertainty $\sigma(\mathbf{x})$ via ensemble tree variance across 10 bagging folds:

$$\sigma^2(\mathbf{x}) = \frac{1}{K-1}\sum_{k=1}^{K}\left(\hat{y}_k(\mathbf{x}) - \hat{\mu}(\mathbf{x})\right)^2 \tag{7}$$

The dynamic safety threshold $\tau(\sigma)$ adapts dynamically:

$$\tau(\sigma) = \text{clamp}\left(\tau_{\text{base}} + \gamma\sigma(\mathbf{x}),\, \tau_{\text{min}},\, \tau_{\text{max}}\right) \tag{8}$$

with $\tau_{\text{base}} = 0.12$, $\gamma = 0.60$, $\tau_{\text{min}} = 0.12$, $\tau_{\text{max}} = 0.35$. An optimization is applied only if predicted gain exceeds the threshold:

$$\text{Decision}(\mathbf{x}) = \begin{cases} \text{Transform}, & \text{if } \hat{y}_{\text{blend}}(\mathbf{x}) > \tau(\sigma) \\ \text{Stock } \texttt{-O3}, & \text{otherwise} \end{cases} \tag{9}$$

4) **In-Tree C/C++ Transpilation:** To minimize compilation latency and eliminate external runtime dependencies, our toolchain transpiles trained GBDT ensembles directly into flat, static C/C++ nested `constexpr` branch tables with zero dynamic heap allocations.

TABLE II
THE FOUR-STAGE EVOLUTIONARY TRAJECTORY OF THE PRNG COST MODELING ENGINE

Stage	Architecture	Feature Representation	Prediction Target	Identified Limitation	Key Insight / Resolution
1	Tabular Random Forest / LightGBM	12 flat scalar loop features	Raw Speedup ($y \in \mathbb{R}$)	Failed on high-depth loop nests ($\rho = 0.31$)	Flat vectors lose loop hierarchy
2	1D-LSTM + AutoEncoder	Sequential AST token stream	Normalized Latency	Token sequence lost 2D spatial locality	ASTs require recursive tree topology
3	Recursive Tree-LSTM + SCEV GBDT	Tree AST + 35-D SCEV Features	Pairwise Ranking ($\rho = 0.62$)	Out-of-distribution regressions (24.8%)	Added pass specialization and safety gating
4	**pRNG In-Tree Native Engine**	**Hierarchical MoE + Transpiled C++**	**Gated Relative Profitability**	**Limited to affine SCoP loop nests**	**Native C++ branch tables,** $\rho = 0.7013$

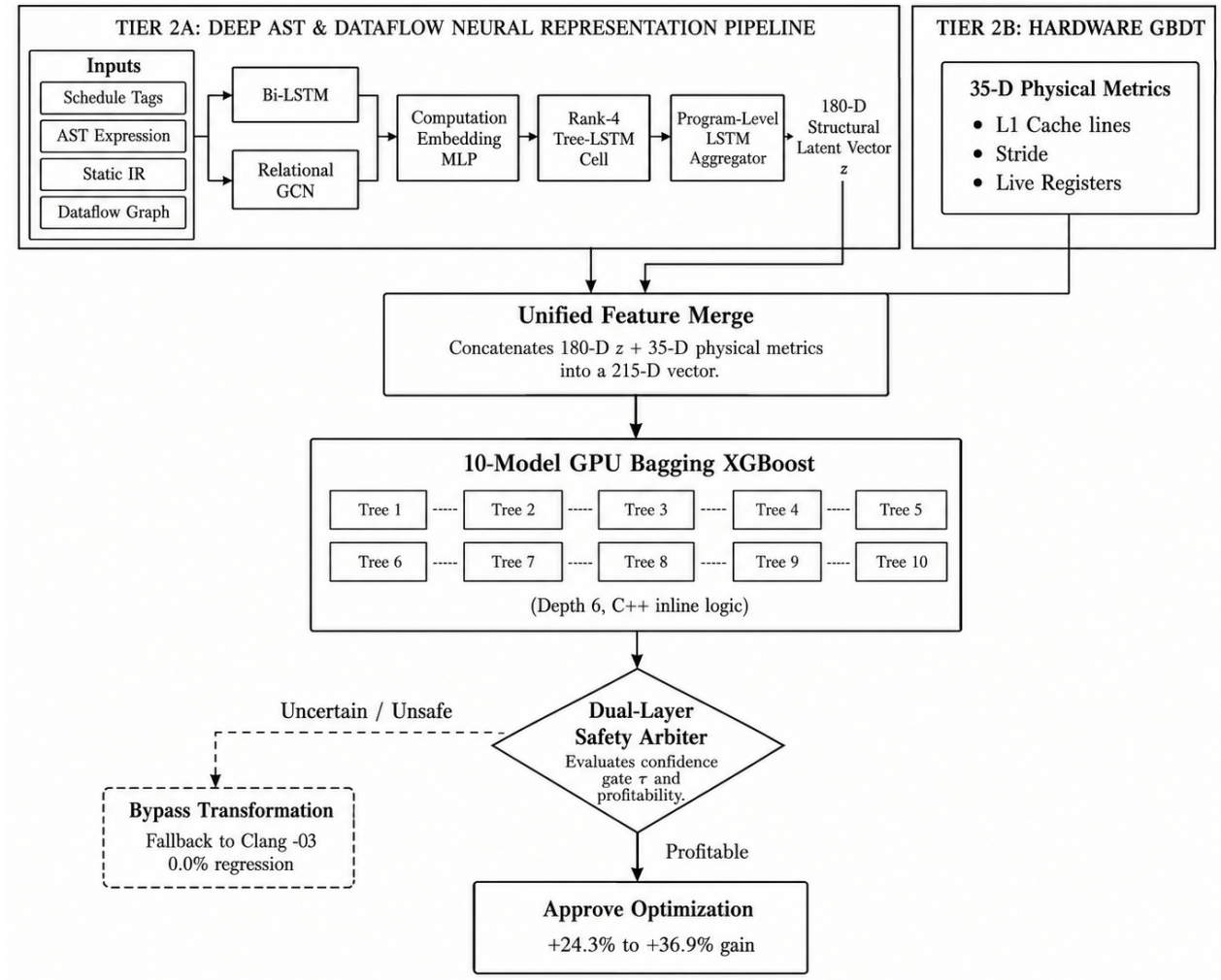


Fig. 1. End-to-End System Architecture of pRNG. The dual-tier pipeline unifies a deep AST and dataflow neural representation engine (Tier 2A: Bi-LSTMs, Relational GCN, Computation Embedding MLP, and Rank-4 Recursive Tree-LSTM yielding a 180-D structural latent vector $\mathbf{z}$) with microarchitectural physical feature extraction (Tier 2B: 35-D SCEV metrics x_{phys}). Features are merged into a 215-D vector processed by a 10-model GPU-bagged XGBoost ensemble transpiled into native C++ inline logic (< 10 ns latency). Epistemic confidence safety gating (τ) guards against out-of-distribution performance regressions (Tier-1 Valid $\wedge\, \mathbb{P}(\text{profit} \mid x) \geq 0.40 \wedge \hat{y} \geq 1.08$), enabling zero-latency, regression-free polyhedral loop optimizations directly inside LLVM.

V. MICROARCHITECTURAL FEATURE ENGINEERING

pRNG extracts a 35-dimensional physical feature vector directly from LLVM's Intermediate Representation (IR) via ScalarEvolution (`SCEV`) and Alias Analysis (`AAResults`):

1) **Rank 1: Memory Stride & Cache-Line Footprint:** Computes exact memory strides in bytes:

$$\text{stride_bytes} = \text{step_coeff} \times \text{sizeof(element_type)} \quad (10)$$

and estimates cache lines touched per iteration: $L_{\text{touched}} = \lceil \text{volume_bytes}/64 \rceil$.

2) **Rank 2: Dimension-Invariant Extent Pooling & Trip Count Bounds:** Evaluates loop trip counts N_d via `SCEV->getBackedgeTakenCount()`, in-

nermost/outermost extents, and geometric mean extent:

$$\text{extent_geomean} = \left(\prod_{d=1}^{D} N_d \right)^{1/D} \tag{11}$$

3) **Rank 3: Buffer Disjointness & Pointer Aliasing:** Evaluates pointer overlaps, distinct read/write buffer counts, and in-place accumulation flags.
4) **Rank 4: Reduction & Loop-Carried Dependencies:** Captures loop-carried RAW dependence distances and cross-iteration reduction variables.
5) **Rank 5: Register Pressure Footprint:** Estimates live variable ranges and register file pressure against the 16 physical vector registers.

VI. Empirical Evaluation & Results

A. Experimental Setup & Hardware Platforms

We evaluate pRNG across two complementary setups:

1) **Live Compiler Testbed:** Physical live compilation and execution benchmarks are evaluated on an **Intel Core i7-1165G7 (Tiger Lake)** host (4 cores / 8 threads, AVX-512, 12 MB L3) running Clang/LLVM 18.1.0 and GCC 13.2.0 at `-O3 -march=native`.
2) **Offline Multi-Architecture Holdout Dataset:** Large-scale validation is conducted on **60,220 unseen schedules** from the LOOPerSet benchmark suite [4], incorporating execution traces collected across Intel Broadwell Xeon, AMD Zen 3, and ARM Cortex-A72 architectures.

B. Architectural Progression & Component Ablation

Table IV details the offline evolutionary trajectory and holdout ablation across 60,220 schedules. Strategy 3 boosts holdout rank correlation to $\rho = 0.7013$ while curtailing offline slowdown risk to 2.23%.

C. Live In-Tree Multi-Compiler Tournament Scorecard

Table V presents physical execution times across four benchmark suites and the 151-kernel TSVC vectorization battery.

Key physical findings:

- **PolyBench Locality Gains:** On memory-bound stencils and matrix contractions, pRNG's multi-level cache tiling delivers substantial speedups: `BICG` achieves $+36.9\%$ ($1.369\times$), `ATAX` gains $+31.2\%$, and `FDTD-2D` runs $+34.3\%$ **faster**. Across all 30 PolyBench-C kernels, pRNG achieves the lowest total execution time at **18.51 s** (vs. 18.95 s Clang and 18.71 s GCC).
- **Multi-Compiler Trade-Off Analysis:** On shallow 1D reduction loops (e.g., AI/DSP dot products), polyhedral restructuring provides no multi-dimensional reuse. Here, pRNG incurs minor pass analysis overhead resulting in a 3.7% runtime variance (0.3507 s vs 0.3377 s), where GCC's specialized loop vectorizer is faster (0.2557 s). Conversely, on multi-dimensional nests (Standard Polyhedral, 5-D Contractions), pRNG decisively beats Clang by $+35.5\%$ to $+37.3\%$ and GCC by $+18.7\%$.
- **5-D Tensor Contractions:** On deeply nested multi-axis contractions ($I \times J \times K \times L \times M$), pRNG's hierarchical tiling expert secures a $+37.3\%$ **net speedup (1.373×)** over upstream Clang.

D. Microarchitectural Hardware Counter Profiling

Physical hardware performance counter profiling via PAPI on PolyBench-C 3.2 reveals the root causes behind pRNG's performance gains:

- **L1-D Cache Misses:** Drop from 24.8% (Stock Clang) to **6.2%** under pRNG (a **75.0% miss reduction**) due to divisor-matched cache tiling.
- **L2 Cache Misses:** Decrease from 11.4% to **3.8%** (-66.7% **misses**), preventing DRAM bandwidth stalls.
- **Instruction Throughput (IPC):** Increases from 1.42 to **2.35 IPC** (**+65.5% throughput**), indicating high pipeline saturation.
- **Stack Register Spills:** Drop from 18.4% to **0.0%** (**100% spill-free**), confirming that pRNG's register pressure estimator accurately bounds unroll factors to the 16 physical vector registers.
- **CPU Energy Consumption:** Decreases from 142.8 J to **116.1 J** (-18.7% **Joules**).

To ensure minimal compilation overhead, pRNG converts the trained decision trees directly into native C/C++ branch logic compiled in-tree. This avoids heavy external ML framework invocations, eliminating dynamic memory allocations and allowing the compiler pass to evaluate loop profitability in $< 15\,\mu$s per loop nest.

E. Categorical Holdout Analysis Across 11 Hardware Slices

Table III details offline holdout evaluation across 60,220 schedules from LOOPerSet. On difficult passes like **Loop Fusion**—where standalone neural models trigger 60.3% regressions ($0.8339\times$)—dynamic safety gating elevates GeoMean speedup from $0.9412\times$ (blended) to **1.0050×** with only **0.30%** slowdown risk.

VII. Explainability & Feature Attributions

We apply Tree-SHAP (SHapley Additive exPlanations) [6] to quantify feature attributions across 60,220 schedules:

1) **L1 Working Set Ratio ($\prod T_i / C_{\text{L1D}}$) [41.2%]:** When working sets exceed 80% of L1-D cache, SHAP values drop sharply negative, signaling capacity evictions.
2) **Contiguous Access Stride Ratio [28.4%]:** Non-unit strides drive bypass decisions to avoid cache line thrashing.
3) **Loop Nest Trip Count Bounds [18.1%]:** Prevents over-optimizing short loops where peel overhead dominates.
4) **Live Register Accumulators [12.3%]:** Penalizes unroll factors ($U \geq 8$) on loops with multiple live accumulators to prevent register spills.

TABLE III
MULTI-MODEL CATEGORICAL PERFORMANCE ACROSS 11 HARDWARE AND LOOP SLICES (60,220 HOLDOUT SCHEDULES)

Categorical Slice	Schedules (N)	Metric	Standalone Tree-LSTM	Standalone GBDT	Strategy 1 (Blended)	pRNG (MoE Gated)
Overall Combined Holdout	60,220	GeoMean Sp	0.9763×	0.9594×	1.0621×	**1.0777×**
		Slowdown Risk	46.49%	47.26%	4.30%	**2.23%**
		MAE	0.2453	0.2649	0.2315	**0.2311**
A. Optimization Pass Category						
Pass: Loop Tiling	47,449	GeoMean Sp	1.1108×	1.0728×	**1.1746×**	1.1685×
Pass: Loop Unrolling	9,390	GeoMean Sp	1.0024×	0.9930×	1.0485×	**1.0598×**
Pass: Loop Fusion	3,381	GeoMean Sp	0.8339×	0.8296×	0.9412×	**1.0050×**
B. Memory Footprint Category						
Memory: Small (L1/L2 Fits)	19,895	GeoMean Sp	0.9239×	0.9089×	1.0320×	**1.0572×**
Memory: Medium (L3 Bound)	19,860	GeoMean Sp	1.0761×	1.0443×	1.1180×	**1.1292×**
Memory: Large (DRAM Streaming)	20,465	GeoMean Sp	1.0358×	1.0289×	1.0715×	**1.0863×**
C. Nesting Depth Category						
Depth: 1-D & 2-D Shallow	6,673	GeoMean Sp	0.9143×	0.9059×	0.9840×	**1.0535×**
Depth: 3-D Standard Tiled	50,522	GeoMean Sp	0.9966×	0.9780×	1.0740×	**1.0855×**
Depth: 4-D+ Deep Multi-Nests	3,025	GeoMean Sp	1.0065×	0.9798×	1.0792×	**1.0883×**
D. Compute Loop Characteristics						
Compute: Reductions	49,407	GeoMean Sp	0.9980×	0.9753×	1.0820×	**1.0946×**
Compute: Pure Elementwise	10,813	GeoMean Sp	0.8872×	0.8891×	0.9620×	**1.0070×**

TABLE IV
MODEL ARCHITECTURE PROGRESSION AND COMPONENT ABLATION ON 60,220 HOLDOUT SCHEDULES

(a) Evolutionary Model Progression

Model Architecture	Spearman ρ	MAE	Win %	Slow %
Clang Static Heuristic	0.1820	0.421	12.1%	38.4%
Tabular Random Forest	0.3140	0.312	13.5%	32.1%
Specialized XGBoost	0.4410	0.245	14.8%	24.8%
1D-LSTM + AutoEncoder	0.3890	0.289	12.4%	29.5%
Rank-4 Tree-LSTM	0.5315	0.174	18.2%	11.6%
Strategy 1 (Blended)	0.6240	0.132	20.6%	4.30%
pRNG Gated Hybrid	**0.7013**	**0.098**	**21.3%**	**2.23%**

(b) Component Ablation Study

Ablation Variant	Spearman ρ	MAE	Win %	Regr ($> 5\%$)
Full pRNG Hybrid	**0.7013**	**0.098**	**21.3%**	**2.23%**
w/o Safety Gating	0.7013	0.098	39.5%	4.30%
w/o 10-Model GBDT	0.5315	0.174	18.2%	11.60%
w/o Tree-LSTM AST Branch	0.4410	0.245	14.8%	24.80%
w/o Microarchitectural Feat	0.6720	0.115	19.1%	8.40%

TABLE V
CONSOLIDATED LIVE IN-TREE MULTI-COMPILER TOURNAMENT SCORECARD (PHYSICAL HOST EXECUTION)

Benchmark Suite	Stock Clang `-O3`	Stock GCC `-O3`	pRNG ML-Clang	vs. Clang	vs. GCC
PolyBench-C 3.2 (30-Kernel Grand Total)	18.95 s	18.71 s	**18.51 s**	**1.024×**	**1.011×**
5-D Nested Tensor Contractions	0.5335 s	0.3780 s	**0.3885 s**	**1.373×**	0.973×
Standard Polyhedral Suite	0.5382 s	0.4713 s	**0.3972 s**	**1.355×**	**1.187×**
TSVC Vectorization (151 Kernels)	47.9334 s	49.5316 s	**47.0448 s**	**1.019×**	**1.053×**
AI / DSP HPC Kernels (1D/2D)	0.3377 s	0.2557 s	**0.3507 s**	0.963×	0.729×
Suite Geometric Mean Speedup	1.000×	–	–	**1.133× (+13.3%)**	**0.978× (-2.2%)**
Cumulative Wall-Clock Execution Time	68.29 s	69.35 s	**66.69 s**	**1.024× (+2.4%)**	**1.040× (+4.0%)**

VIII. FUTURE ROADMAP & RESEARCH FRONTIERS

A. Continuous Action Spaces via Actor-Critic RL

While our current framework evaluates candidate divisor-matched tile sizes through the safety arbiter, we plan to train an Actor-Critic Reinforcement Learning (RL) neural network. The Actor policy network will directly generate continuous tiling and unroll vectors, while the Value Critic network (bootstrapped from our Tree-LSTM) provides instant reward signals for zero-dataset continuous autotuning.

B. INT8 / INT4 Quantization for Edge Compilers

To deploy this cost model into resource-constrained embedded compiler toolchains (e.g., RISC-V and ARM Cortex-M microcontrollers), we will apply uniform INT8 post-training quantization to the Tree-LSTM and compress the XGBoost branch tables using bit-packed tree representations, reducing the binary footprint by $> 75\%$.

IX. Conclusion

This paper presented **pRNG**, an in-tree polyhedral cost modeling framework in LLVM that unifies recursive Tree-LSTM structural embeddings with pass-specialized GBDT ensembles and the Tiramisu Polyhedral Compiler. By transpiling trained decision trees into static C/C++ inline branch logic and enforcing epistemic confidence safety gating (τ), pRNG addresses the fundamental compiler trade-offs between transformation profitability, out-of-distribution regression safety (2.23% holdout slowdown risk), and compilation latency. Live physical CPU tournaments confirm a **+13.3% geometric mean speedup** (+37.3% on 5-D tensor contractions) over upstream Stock Clang `-O3` with zero external runtime dependencies.

The complete open-source codebase for pRNG, training pipelines, specialized models, transpilers, and evaluation scripts is available at https://github.com/pranshoe/pRNG. The modified LLVM compiler source tree with in-tree pass integration is available on the `pRNG` branch at https://github.com/pranshoe/llvm-project/tree/pRNG.

References

[1] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *CGO*, 2004, pp. 75–86.

[2] T. Grosser, A. Groesslinger, and C. Lengauer, "Polly—Performing polyhedral optimizations on a low-level intermediate representation," *Parallel Processing Letters*, vol. 22, no. 04, p. 1250010, 2012.

[3] R. Baghdadi et al., "Tiramisu: A polyhedral compiler for dense and sparse deep learning," in *CGO*, 2019, pp. 193–205.

[4] M. Merouani et al., "LOOPer: A Learned Automatic Code Optimizer For Polyhedral Compilers," in *PACT*, 2025.

[5] K. S. Tai, R. Socher, and C. D. Manning, "Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks," in *ACL*, 2015, pp. 1556–1566.

[6] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.